Lecture Note 6: Resource Allocation and Deadlocks

CSS225 Operating Systems

School of ICT, SIIT, Thammasat University

Outlines

1. Introduction to Resource Allocation and Deadlocks
2. Deadlock Prevention
3. Resource-Allocation Graph
4. Resource-Allocation Graph for Deadlock Detection
5. Resource-Allocation Graph for Deadlock Avoidance
6. Banker's Algorithms
7. Safety Algorithm
8. Resource-Request Algorithm for Deadlock Avoidance
9. Deadlock-Detection Algorithm for Deadlock Detection
10. Recovery from Deadlock

1. Introduction to Resource Allocation and Deadlocks

Analogy: A Whiteboard Situation

A teacher (OS) need to give pens to 2 students (processes: P1 and P2):

- **Task**: Each student (P1, P2) needs 2 pens: 1 red pen and 1 blue pen to finish a task.
- **Allocation**: Currently, P1 owns 1 red pen while P2 owns 1 blue pen.
- **Available**: There are one free red pen at the teacher.
- **Resource-allocation problem**: How can the teacher the pen(s) to the students such that all students will finish their tasks?

Situation of Interest: Resource Allocation

1) **What is Resource Allocation?**
   a) Resource allocation is the mechanism of distributing system resources among processes in a way that ensures efficient execution and prevents conflicts.
      - Meaning = Allocating/giving resources to processes.
   b) Resource allocation is done by the OS.
   c) Examples of these resources include CPU cycles, memory, storage, I/O devices, network bandwidth, variables, files, databases, etc.
      - Ex: To run a process, the OS must allocate a space in the main memory to store this process and the CPU time to run it.

Situation of Interest: Resource Allocation

2) Why is Resource Allocation Important?

- Requirement: Each process requires to access/own resources to finish its task and terminate.
  - We have the following steps: A process → has a task to do → owns a set of resources → finishes its task and terminate.
  - When a process finishes its task, this process returns all owned resources to the system.
- Contradiction: We have limited resources but a large number of processes run concurrently.
- Challenging: The OS needs to allocate the shared and limited resources to these processes such that all of them can finish their jobs and terminate.

Situation of Interest: Conclusion

In a system, multiple processes compete for a set of resources.

1) Each process has a task/job to do/complete.
2) To execute its job, a process must acquire/access/own the necessary resources.
3) The operating system (OS) allocates available resources to each process and ensures that all processes can complete their tasks/jobs efficiently. (Centralized allocation)
4) When a process finishes its job, it releases all allocated resources back to the system and terminates.
   - This allows other processes to access these resources.

Example: Allocating Resources to Processes

Scenario: Consider a system where three processes (P1, P2, and P3) require access to three types of resources:

- CPU time
- Memory (RAM)
- Printer

Limited Resources (Constraints): The system has limited resources:

- 2 CPUs
- 4 GB RAM
- 1 Printer

Example: Allocating Resources to Processes

Step 1: Resource Requirement by Processes.

Each process requests resources based on its requirements:

- P1 needs 1 CPU, 2 GB RAM, and 1 printer.
- P2 needs 1 CPU, 2 GB RAM, and 1 printer.
- P3 needs 2 CPUs, 1 GB RAM, and 1 printer.

A strategy to give resources to processes such that each process will finish its job, one by one (sequentially).

Step 2: Sequentially Allocating Resources

The operating system (OS) uses a resource allocation strategy to distribute the available resources:

- P1 gets 1 CPU, 2 GB RAM, and the printer.
- P2 gets 1 CPU, 2 GB RAM, but must wait for the printer to be released by P1.
- P3 must wait since there are no available CPUs.

Example: Allocating Resources to Processes

Step 3: Execution, Resource <u>Release and Re-allocation</u>

a) P1 completes execution and releases its allocated resources (1 CPU, 2 GB, printer).
   - All resources are returned to the system for future use.

b) The OS gives the printer to P2 such that P2 can use the printer. As a result, P2 finishes its job and terminates.
   - All resources are returned to the system for future use.

c) The OS gives 2 CPUs, 1 GB, the printer to P3. As a result, P3 finishes its job and terminates.
   - All resources are returned to the system for future use.

<u>Important</u>: The selection of the next process (that the OS will give resource to) can be determined by using the Banker's algorithms.

Problem Statement: Resource-Allocation Problem

1) **Given Scenario**: The OS must allocate resources to processes such that they will finish their tasks. This current state of the system will be given and denoted by the following parameters:
   - n (Processes): The total number of processes competing for resources.
   - m (Resource Types): The total quantity of available resource types (CPU, Memory, I/O)
   - Max Demand: The number of resources that each process must own to complete its task/job.
   - Allocation: The number of resources currently owned by each process.
   - Need: The remaining resources required by a process (Max Demand - Allocation).

Problem Statement: Resource-Allocation Problem

2) **Problem:** Given the current state of the system, can the OS allocate limited resources so that every process can eventually finish and terminate?

3) **Key Considerations:**

   a) **Sequential Allocation:** The OS will give resources to each process, one by one. We must determine this order of processes that the OS will give resources to.
      - For example, consider 3 processes (P1, P2, P3). Here, the OS will give resources to P2, then, P1, and, lastly, P3 in this order such that all processes will finally finish their tasks. This order can be denoted as <P2, P1, P3>.

   b) **Feasible Outcomes:** There are 2 possible outcomes/results.
      - **The Safe State:** The OS can allocate the resources such that all processes will finish their tasks. The order of processes (that the OS allocates the resources to) is called a **safe sequence**.
      - **Deadlock:** If we cannot find a safe sequence, we say that the current state of the system is in a deadlock (= unsafe state). As a result, a circular wait occurs and no process can ever finish.

Problem Statement: Limited Resources

Consider there are many processes waiting for resources. There are 2 general cases as follows.

1) If sufficient resources are available, the OS may allocate them to all processes simultaneously. All processes will finish their tasks.
2) If resources are limited (not enough for all process at once), the OS may allocate them sequentially, allowing processes to complete one at a time.
   a) Resource allocation graph
   b) Banker's algorithms

Problem Statement: What we are finding in Resource Allocation Problems

Consider a limited-resource situation, from a resource-allocation problem, the OS must allocate a set of resources to each process sequentially (one-by-one). We will:

a) Determine whether the system is in a safe state or a deadlock.

b) If the system is in a safe state, we will find a safe sequence.

Resource Allocation Mechanisms: General Concept

The OS must allocate the resources to n processes (called the current state of the system).

1) Based on the available resources, the OS selects a process and allocates resources to it.
   - The criteria: Selecting a process who receive these resources and will finish its task.
2) Once the process completes its job, it releases all allocated resources back to the system.
3) Repeat Steps 1 and 2.
4) The resource allocation process stops in the following cases:
   a) If all processes successfully complete their jobs and terminate, the system is considered to be in a safe state. The sequence in which processes terminate is called the safe sequence.
   b) In the following situation, a deadlock occurs:
      i. The OS allocates resources to a set of processes, but there are not enough available/remaining resources to allow any more processes to complete.
      ii. These processes remain unfinished as they are waiting for additional resources.
      iii. In this state, the system is considered to be in a deadlock.

Resource Allocation Mechanisms: Key Rules

1. Process-termination conditions:
   - Each process has a task to do. A process will finish its task if the process owns all required resources.
2. Release allocated resources and return them to the system:
   - Once a process finishes its job, it releases all allocated resources back to the system and terminates.
   - As a result, the OS can reallocate these resources to other processes.
   - That is a reason on why using the OS applies sequential resource allocation and can let all processes finish their tasks even trough we have limited resources.

Two Types of System States: Safe State vs. Deadlock

There are 2 types of system states. A system will be either in a safe state or a deadlock.

1) **Safe State**: A system is in a safe state if the operating system can allocate resources in such a way that every process eventually gets the required resources, finishes its job, and terminates.
   - **Safe Sequence**: A safe sequence is a list of processes who have finished their jobs in order.

Safe State ←→ Safe Sequence

2) **Deadlock**: A deadlock occurs when a set of processes are stuck in a waiting state, each holding some resources while simultaneously waiting for additional resources that are currently allocated to other waiting processes. In this situation, no process can proceed, leading to an indefinite halt in execution.

Example: Meaning of a Safe Sequence

Consider a resource-allocation problem consisting of 4 processes. Assume that the system is in a safe state and has the following safe sequence <P2, P1, P4, P3>. What is the meaning of this safe sequence?

Ans: A safe sequence list an order of processes who have acquired all required resources and then terminated. The safe sequence <P2, P1, P4, P3> refers that:

- First, P2 has owned all required resources, finished its job, returned all owned resources, and then terminated.
- Second, P1 has owned all required resources, finished its job, returned all owned resources, and then terminated.
- Third, P4 has owned all required resources, finished its job, returned all owned resources, and then terminated.
- Lastly, P3 has owned all required resources, finished its job, returned all owned resources, and then terminated.

Terms: Resource Types and Instances

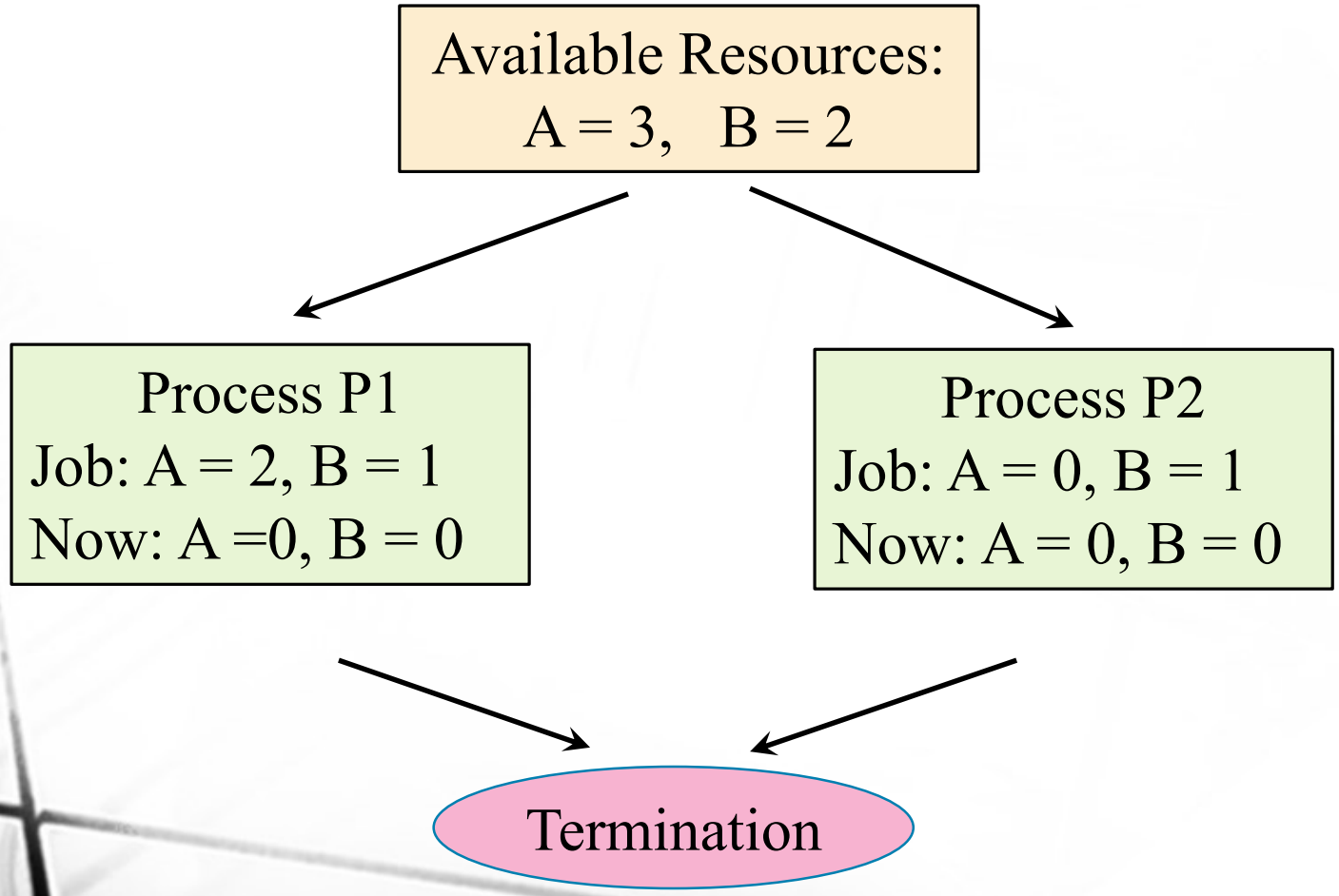
Processes compete for a limited number of resources in a computer system. These resources are categorized into resource types, where each type consists of multiple identical instances.

For example:

- A printer resource type may have 5 printer instances available in the system, allowing up to 5 processes to acquire a printer at the same time.
- A CPU resource type may have 3 CPU instances available, allowing up to 3 processes to acquire a CPU simultaneously.

Example: Safe State – Case 1 Simultaneous Allocation

Determine whether the system is in a safe sate and find a safe sequence.



Resource types: A and B

In this scenario, we have enough A and B to give to P1 and P2 (fulfill the job requirement) at the same time.

Steps:

1. Give P1: 2 of A and 1 of B.
2. Give P2: 0 of A and 1 of B.
3. P1 and P2 will finish their jobs and terminate.
4. The system is in a safe state.

Example: Safe State – Case 2 Sequential Allocation

Determine whether the system is in a safe sate and find a safe sequence.

Selecting a process such that it will finish its task and terminate.

Available Resources:
A = 3, B = 2

Process P1
Job: A = 2, B = 1
Now: A =0, B = 0

Process P2
Job: A = 2, B = 2
Now: A =0, B = 0

Termination

1. Give P1: 2 of A and 1 of B.
2. P1 finished and terminated. P1 will return all resource to the system.
3. Give P2: 2 of A and 2 of B.
4. P2 finished and terminated. P2 will return all resource to the system.
5. Both processes terminated.
6. The system is in a safe state and the safe sequence is

   <P1, P2>.

Example: Safe State – Case 3 Sequential Allocation

Determine whether the system is in a safe sate and find a safe sequence.

Selecting a process such that it will finish its task and terminate.

Available Resources:
A = 3, B = 2

Process P1
Job: A = 2, B = 1
Now: A =0, B = 0

Process P2
Job: A = 2, B = 2
Now: A =0, B = 0

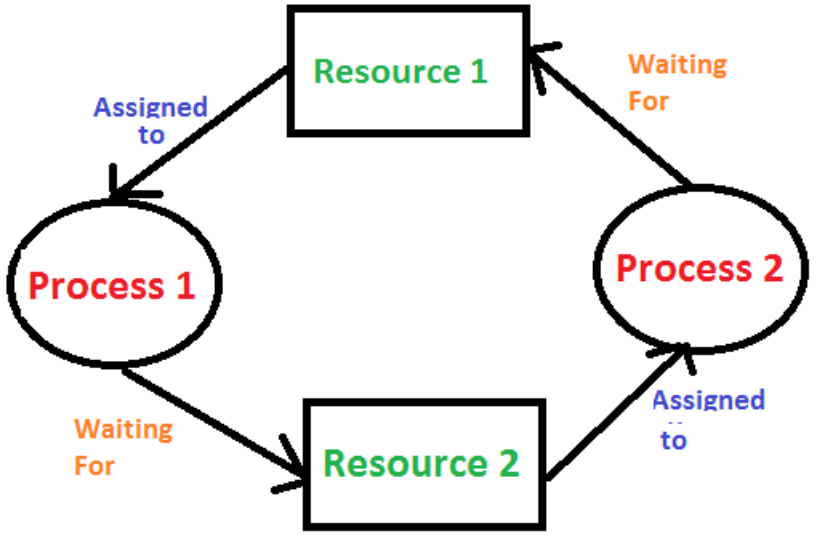
Termination

1. Give P2: 2 of A and 2 of B.
2. P2 finished and terminated. P2 will return all resource to the system.
3. Give P1: 2 of A and 1 of B.
4. P1 finished and terminated. P1 will return all resource to the system.
5. Both processes terminated.
6. The system is in a safe state and the safe sequence is

   <P2, P1>.

A Deadlock Problem in Resource Allocation

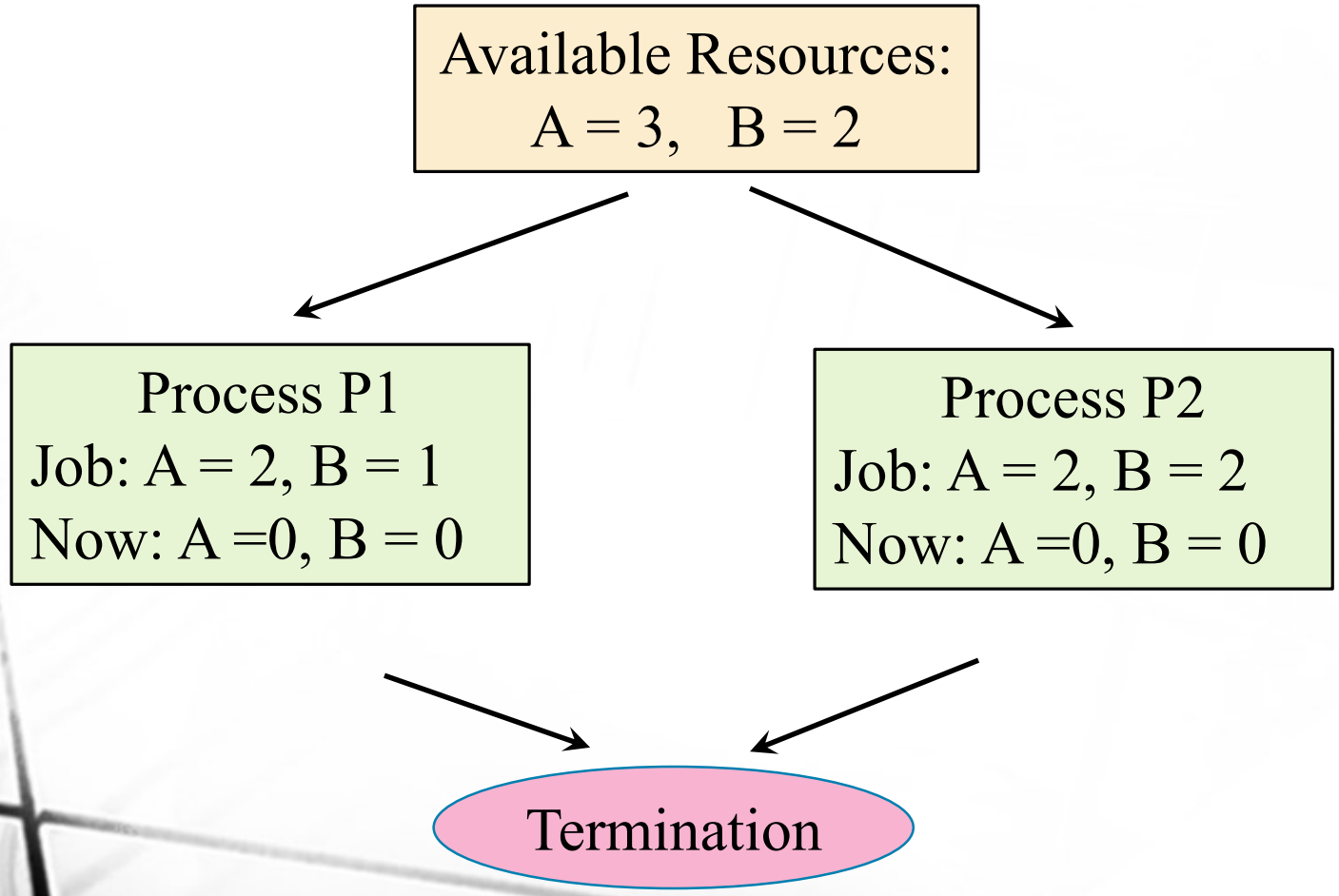
1. Since the number of resources is limited (not infinity), if the OS does not allocate the resources properly, a deadlock (in resource allocation) could happen.
2. A deadlock (in resource allocation) is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.
   - There are no more available/free resources left in the system.
   - A deadlock keeps all processes waiting forever (no progress, could not finish their jobs, and cannot terminate).
   - For example, in the below diagram, Process 1 is holding Resource 1 and waiting for Resource 2 which is acquired by Process 2, and Process 2 is waiting for Resource 1.



Example: Deadlock

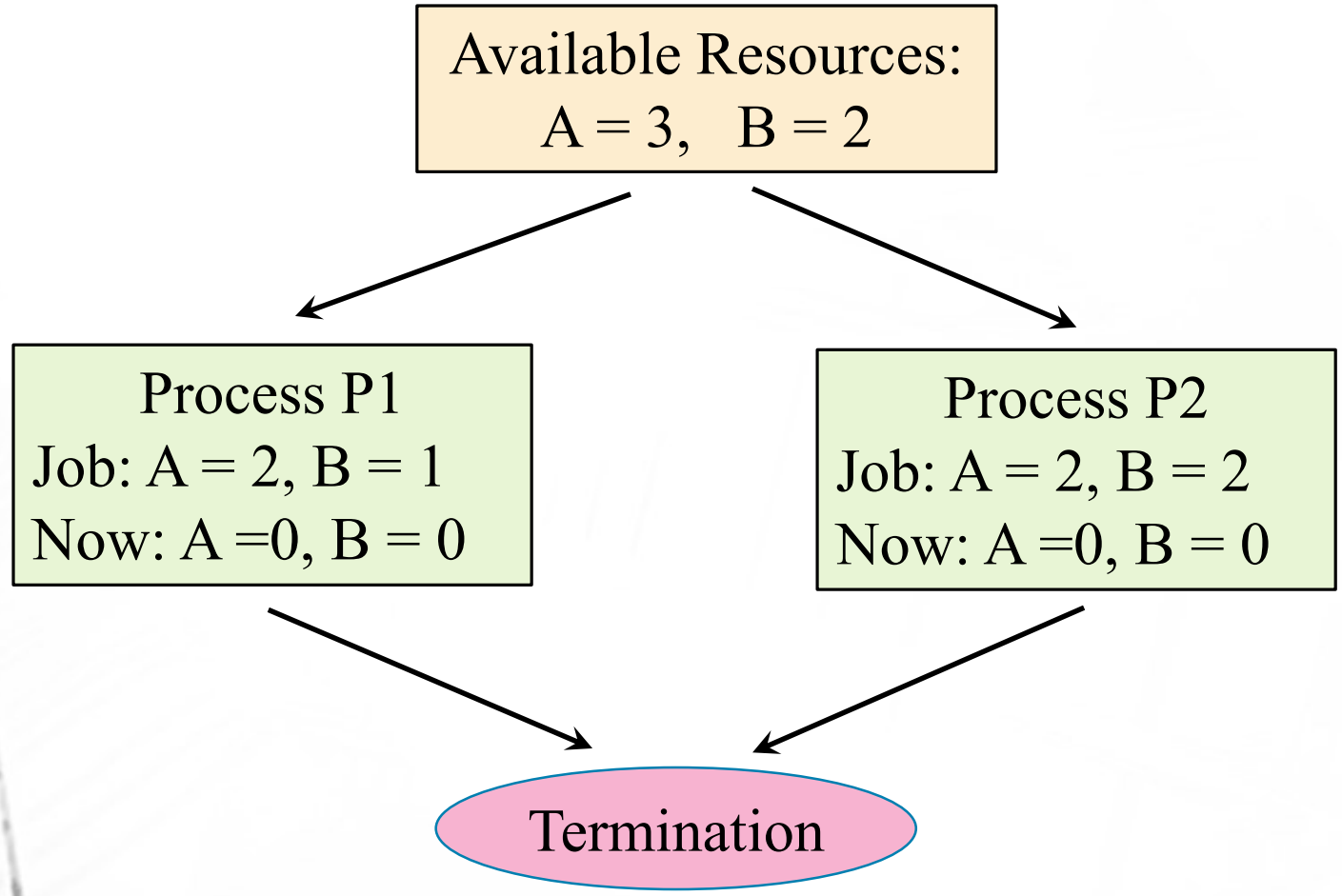
This example shows a bad resource-allocation strategy.

Determine whether the system is in a safe sate and find a safe sequence.



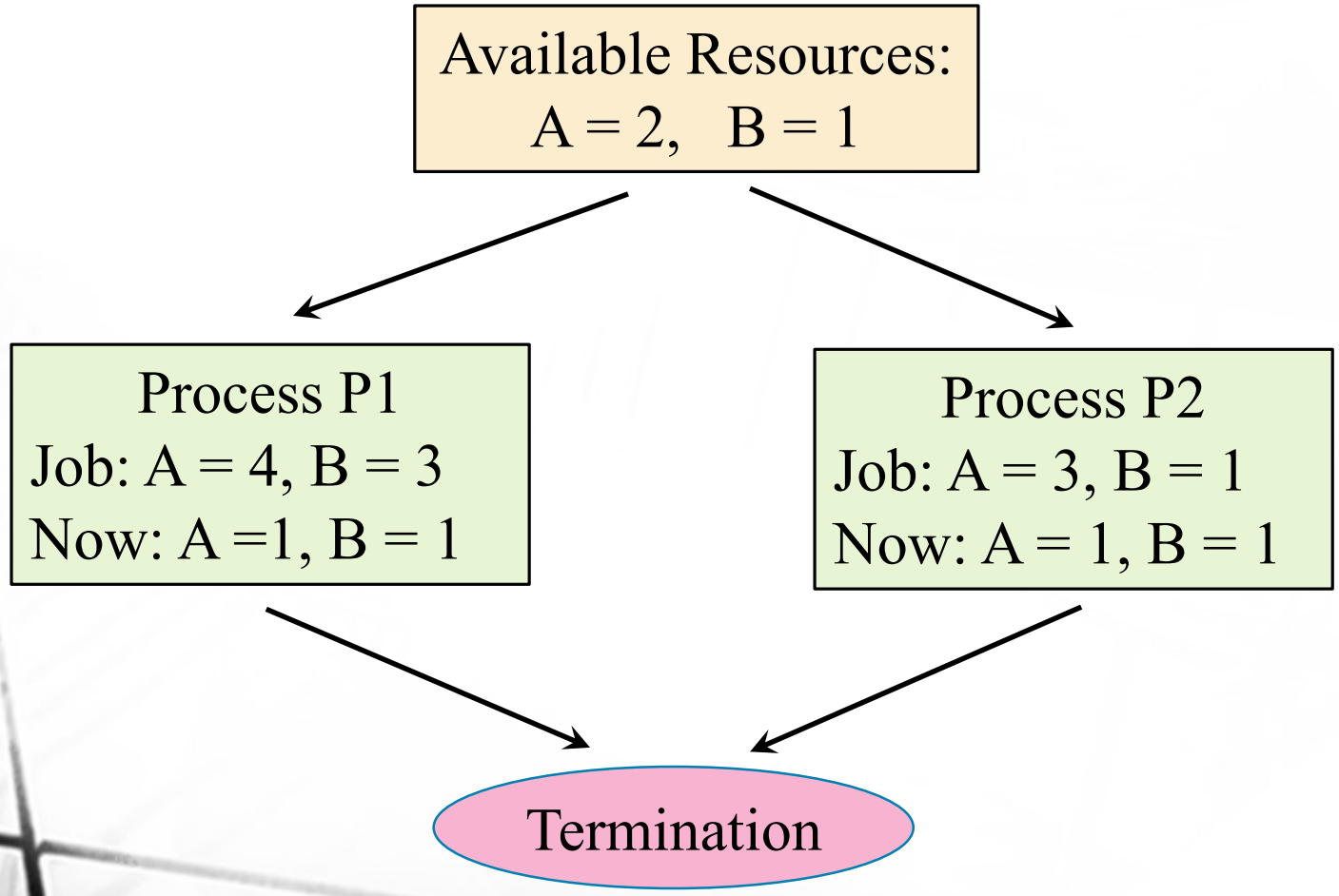
1. Give P1: 1 of A and 1 of B.
2. Give P2: 2 of A and 1 of B.
3. P1 is waiting for 1 of A.
4. P2 is waiting for 1 of B.
5. Both processes cannot finish their jobs and they are in a deadlock.

Example: Can we have a different allocation strategy such that the system will be in a safe sate?



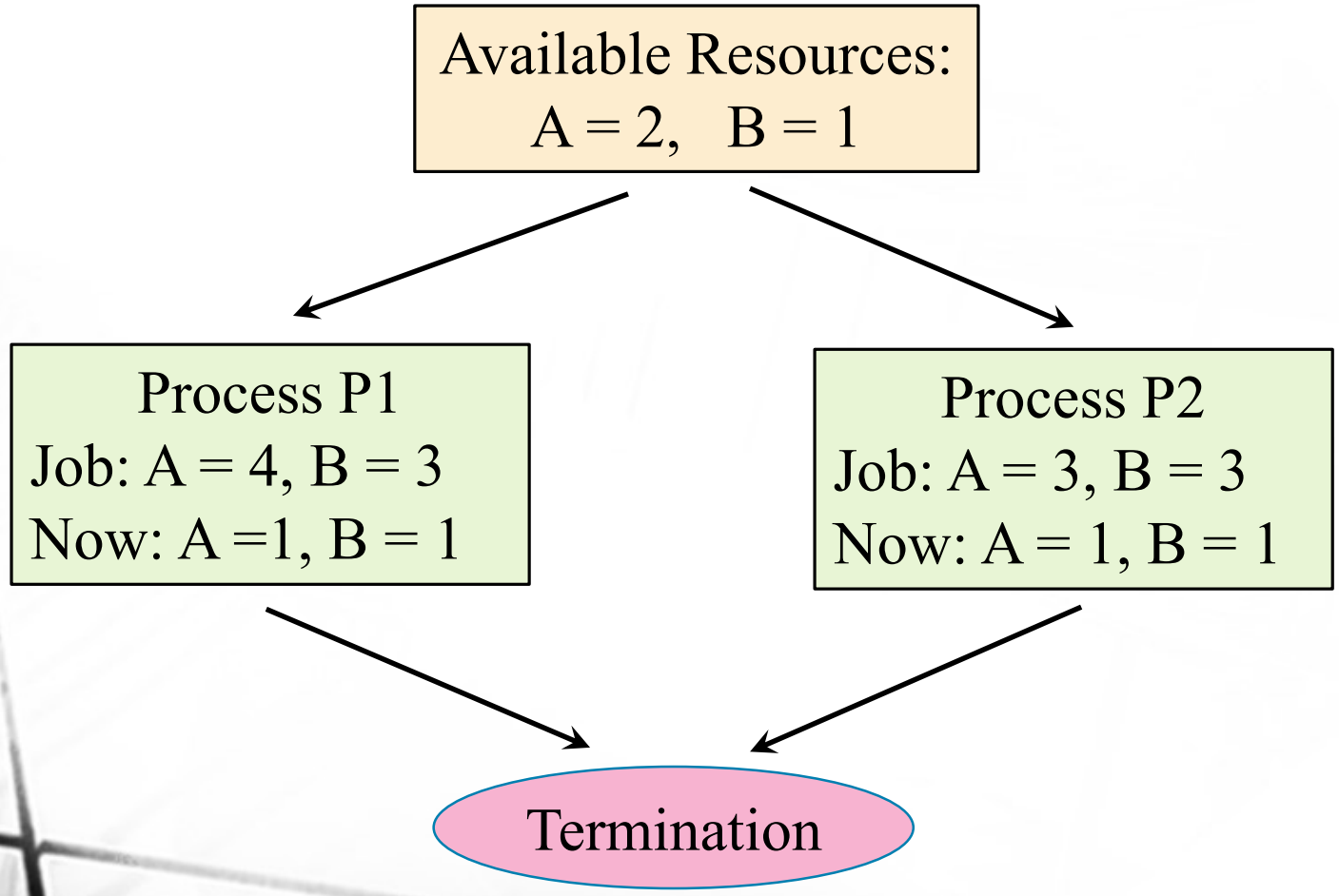
Example: Safe State or Deadlock – Case 1

Consider the following scenario. Determine whether the system is in a safe state or a deadlock. If the system in a safe state, find a safe sequence.



Example: Safe State or Deadlock – Case 2

Consider the following scenario. Determine whether the system is in a safe state or a deadlock. If the system in a safe state, find a safe sequence.



Necessary Condition vs. Sufficient Condition

- Necessary condition:
  - If A is a necessary condition of B: when B happens, we have A. (But the reverse is not true: Having A does not guarantee B happening.)
- Sufficient condition:
  - If A is a sufficient condition of B: when we have A, B happens. (But the reverse is not true. If B happens, we might not have A.)

Necessary Conditions for Deadlock

A deadlock can/might happen in the system if the following conditions hold simultaneously.

1. *Mutual exclusion*: At least one resource is not a shareable; only one process at a time can use a resource. If this resource is owned, other processes would like to use this resource must wait.
2. *Hold and wait*: A process holding at least one resource is waiting to acquire additional resources held by other processes
3. *No preemption*: A resource can be released only voluntarily by the process holding it, after this process has finished its job.
4. *Circular wait*: Given a set of n processes $\{P_1, P_2, \ldots, P_n\}$ waiting for resources. The circular wait means that:

P_1 is waiting for the resource held by P_2 and

P_2 is waiting for the resource held by P_3 and

$\vdots$

P_n is waiting for the resource held by P_1

Methods of Handling Deadlocks

Operating systems can deal with the deadlock problem in one of three ways:

1. They can use a protocol to *prevent* (Sect. 6.2) or *avoid* (Sect. 6.4 and 6.8) deadlocks, ensuring that the system will never enter a deadlocked state.
2. They can allow the system to enter a deadlocked state, *detect* (Sect. 6.5 and 6.9) it, and *recover* (Sect. 6.10).
3. They can ignore the problem altogether and pretend that deadlocks never occur in the system.

Note that the third solution is the one used by most operating systems, including Linux and Windows. It is then up to the application developer to write programs that handle deadlocks.

Tools to Determine a Safe State or a Deadlock

1. Resource-Allocation Graph
   a) Resource-allocation graph
   b) Wait-for graph
2. Banker's Algorithms
   a) Safety algorithm
   b) Resource-request algorithm
   c) Deadlock-detection algorithm

2. Deadlock Prevention

We can prevent the occurrence of a deadlock by making sure that, at least, one of the necessary conditions cannot happen.

Examples of Deadlock Prevention

The following are some ideas with limited usages:

1. **No mutual exclusion**:
   - There will be no mutual exclusion if all resources are sharable. Many processes can access the resources without waiting.
   - Each resource types provided enough instances to all processes.
2. **No hold and wait**:
   - Creating protocols such that, whenever a process requests a resource, it does not hold any other resources. For example.
     - 1st protocol: Each process must request and be allocated all its needed resources before it begins execution.
     - 2nd protocol: A process will request some resources only when it has none.

Examples of Deadlock Prevention

3. **Allow preemption**:
   - If a process is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the process must wait), then all resources the process is currently holding are preempted. In other words, these resources are implicitly released.

Examples of Deadlock Prevention

4. **No circular wait**:
   - Assign each resource type a number. A process can request an instance in a resource type if the requested resource-type number is higher than the currently held resource type-number(s).
   - For example,
     - Let $F: R \rightarrow N$ is a function mapping from a resource type to a number.

$$F(\text{tape drive}) = 1$$
$$F(\text{disk drive}) = 5$$
$$F(\text{printer}) = 12$$

     - Consider that a process is holding an instance of the resource type R_i.
     - This process can request an instance in the resource type R_j if and only if $F(R_j) > F(R_i)$.

3. Resource-Allocation Graph

- We will use a graph to represent the current state of the resource allocation.
- From the given graph, we find whether the system is in the safe state or the deadlock.
- It is applicable to a simple case where a process needs only one instance of each resource type (i.e., it might need several resource types but need only one instance per each resource type).

Resource-Allocation Graph

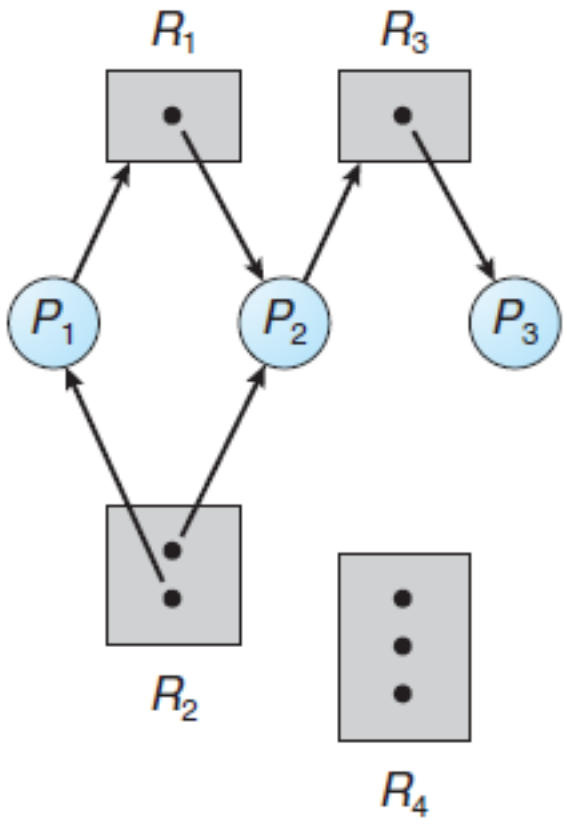
For a simple case, we can use a resource-allocation graph to study whether a deadlock happens. A resource-allocation graph consists of

- A set of *vertices* is divided into 2 subsets:
  - A set of n processes $\{P_1, P_2, \dots, P_n\}$
  - A set of m resource types $\{R_1, R_2, \dots, R_m\}$
    - Each resource type might consist of 1 instance or many instances.
- A set of *edges*:
  - A *request edge* $P_i \rightarrow R_j$ indicates that the process P_i has requested an instance of the resource type R_j and is currently waiting for that instance.
  - An *assignment edge* $R_j \rightarrow P_i$ indicates that an instance in the resource type R_j has been allocated to the process P_i.
  - Note that a request edge will point to only the rectangular (resource type) while the assignment edge will start from the dot (instance).

Example: Resource-Allocation Graph

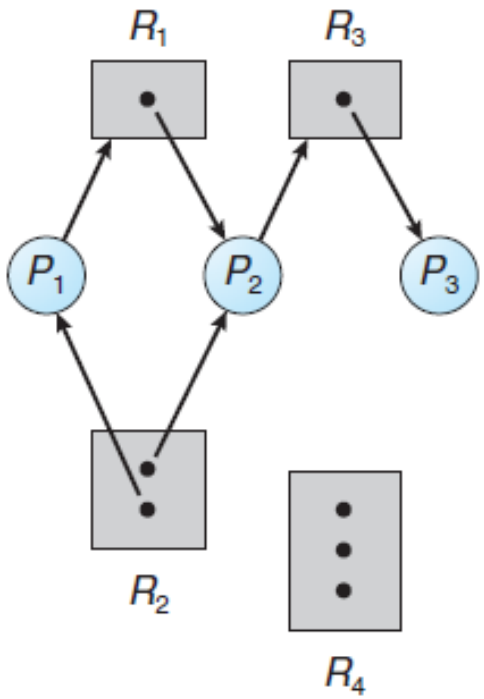
- Notation:
  - A circle represents a process.
  - A rectangular represents a resource type.
  - A dot is an instance.
- The sets of P, R, and E
  - $P = \{P_1, P_2, P_3\}$
  - $R = \{R_1, R_2, R_3, R_4\}$
  - $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$
- Resource instances:
  - 1 instances in the resource type R_1
  - 2 instances in the resource type R_2
  - 1 instances in the resource type R_3
  - 3 instances in the resource type R_4

Equivalent: Sets ←→ Graph



Example: Resource-Allocation Graph

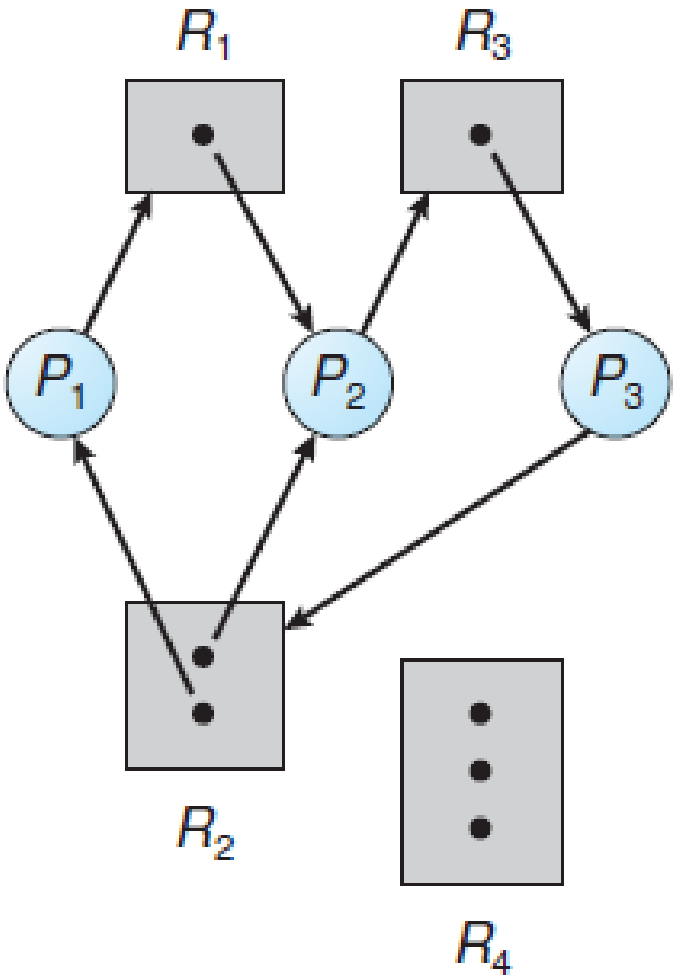
- Meaning:
  - Process P_1 is holding an instance of resource type R_2 and is waiting for an instance of resource type R_1.
  - Process P_2 is holding an instance of R_1 and an instance of R_2 and is waiting for an instance of R_3.
  - Process P_3 is holding an instance of R_3.



- Note that a resource-allocation graph can be represented by
  - a graph
  - or a set of P, R, and E

Example: Resource Allocation Graph

1) Find the sets of P, R, and E.



2) How many instances are in each resource type?

4. Resource-Allocation Graph for Deadlock Detection

Problem Statement

Given a resource-allocation graph (representing the resource allocation in the system):

1) Determine whether the system is in a safe state or a deadlock.
2) If the system is in a safe state, find a safe sequence.

Check Deadlocks by Using the Graph

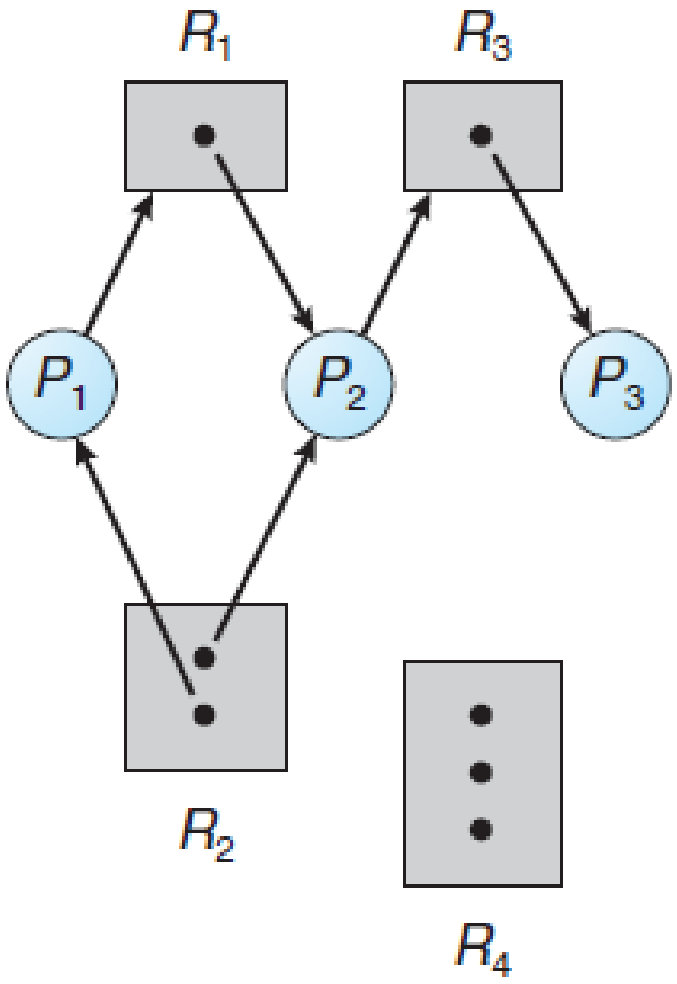
How could we know this?

We follow these steps to determine the system's state:

1. Select a process from the resource allocation graph.
2. Check if the selected process has all the required resources to complete its execution:
   - **If yes**, the process completes its job, releases all its owned resources back to the system, and terminates.
   - **If no**, select another process and repeat Step 2.
3. After evaluating all processes in the graph:
   - **If all processes terminate**, the system is in a safe state, and the order in which processes complete is called the safe sequence.
   - **If some processes remain unable to terminate**, the system is in a deadlock.

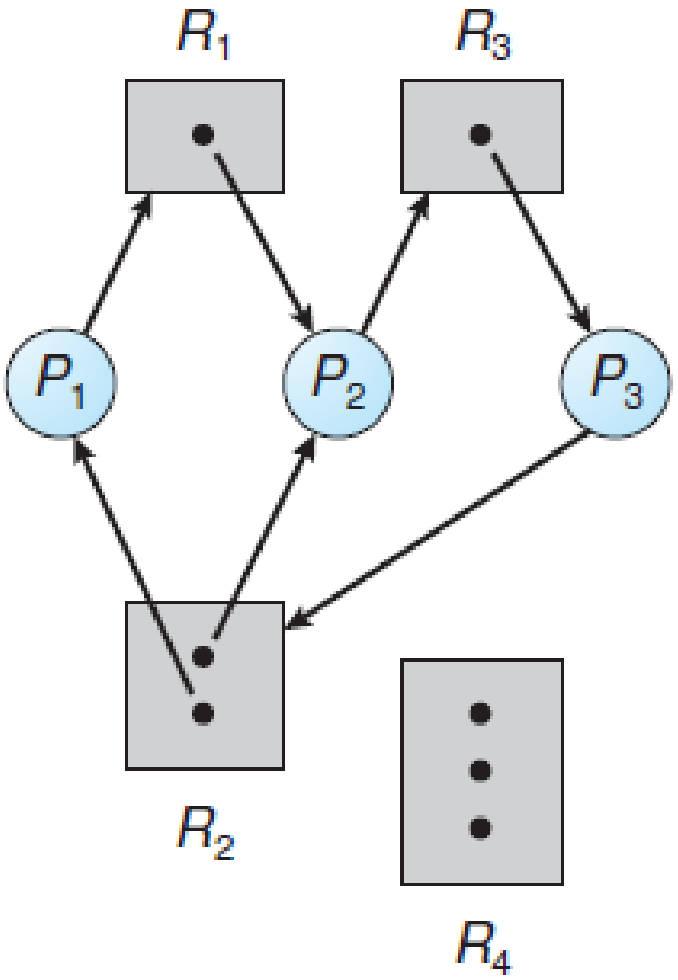
Example: Safe State or Deadlock – Case 1

Consider a resource allocation problem shown by the following resource allocation graph. Determine whether the system is in a deadlock or a safe state. If the system in a safe state, find a safe sequence.



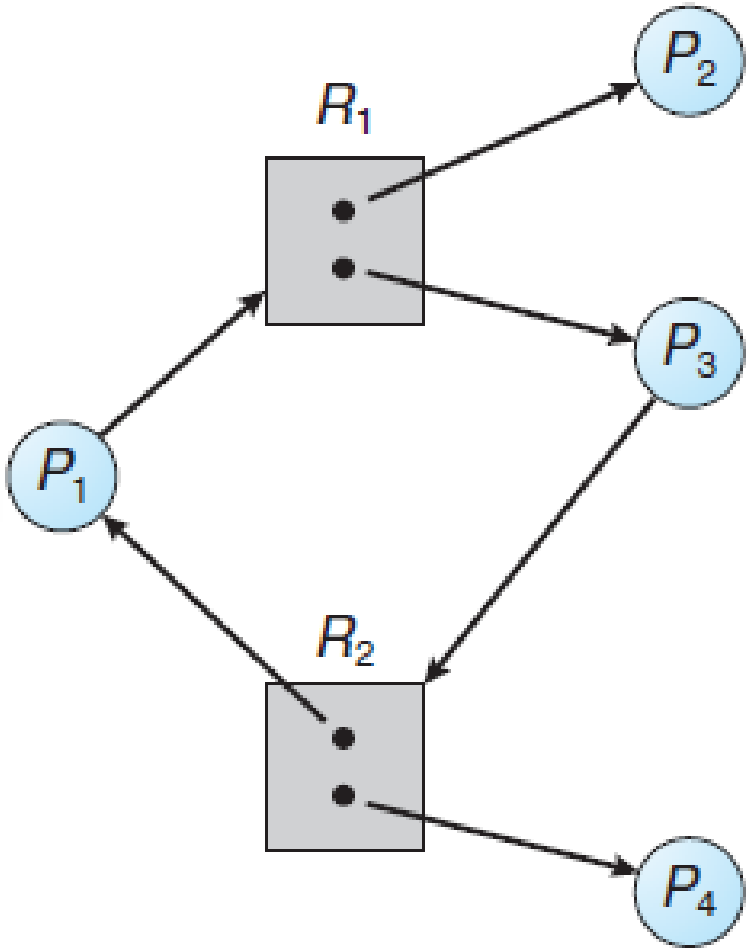
Example: Safe State or Deadlock – Case 2

Consider a resource allocation problem shown by the following resource allocation graph. Determine whether the system is in a deadlock or a safe state. If the system in a safe state, find a safe sequence.



Example: Safe State or Deadlock – Case 3

Consider a resource allocation problem shown by the following resource allocation graph. Determine whether the system is in a deadlock or a safe state. If the system in a safe state, find a safe sequence.



Example: Safe State or Deadlock – Case 4

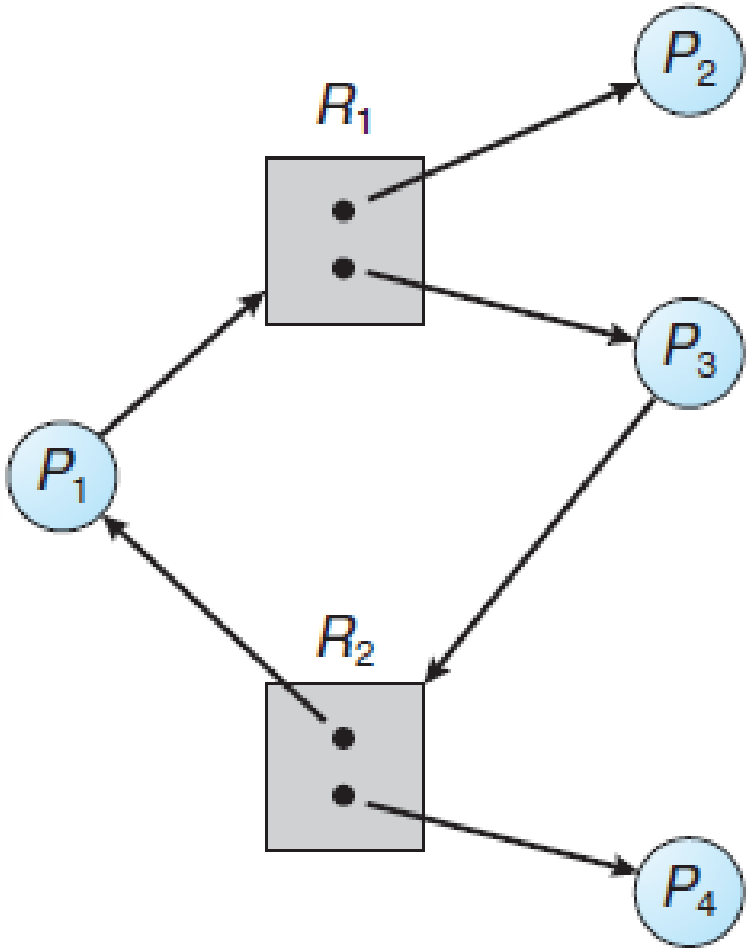
Consider a resource allocation problem shown by the following resource allocation graph.

a) Is <P4, P2, P1, P3> a safe sequence?

b) Is <P4, P2, P3, P1> a safe sequence?

c) Is <P2, P4, P3, P1> a safe sequence?

d) Is <P1, P2, P3, P4> a safe sequence?

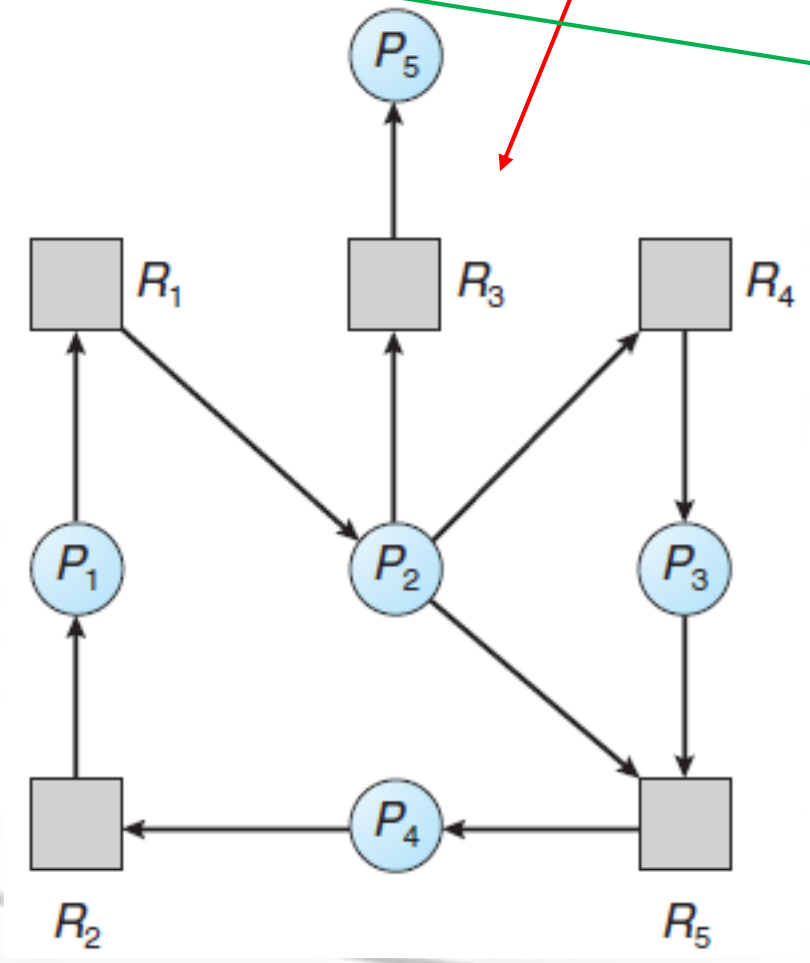


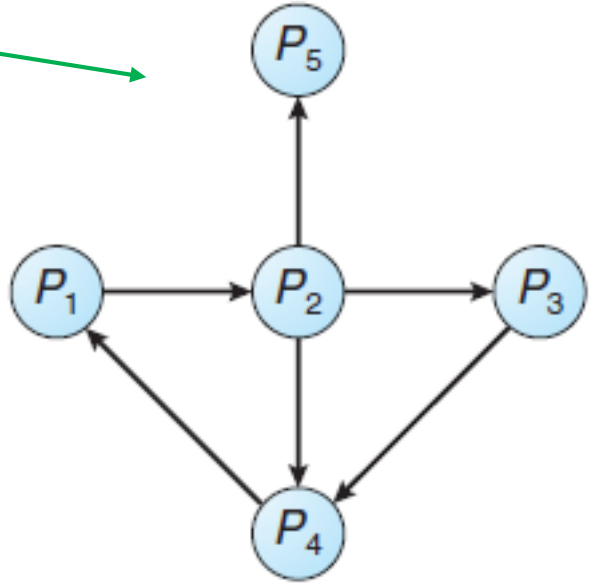
Wait-For Graph

1) If all resource types have only one instance, we can use a wait-for graph to detect deadlocks.
2) A wait-for graph is a variant of the resource-allocation graph.
    1) A wait-for graph will show only process nodes.
3) We can change a given resource allocation graph to be a wait-for graph by using the steps:
    1) We remove the resource nodes.
    2) We connect/link appropriate edges together.
4) If the wait-for graph contains a cycle, there is a deadlock in the system.

Example: Applying a Wait-For Graph

Given a resource-allocation graph on the left-hand side, we can draw the corresponding wait-for graph as shown on the right-hand side. As seen, there are deadlocks





5. Resource-Allocation Graph for Deadlock Avoidance

Additional Assumption for Deadlock Avoidance

If a process wants to acquire an instance of a resource type, the following steps are followed:

1) The process makes a request to acquire the resource.
2) The operating system evaluates and decides whether to grant the request.
3) If granted, the process is allocated an instance of the resource and gains control over (or owns) it.
4) Once the process has finished its job (by using the resource), it releases the instance to the system (and other processes can use it).

Problem Statement and Criteria

Problem Statement:

When a process requests ownership of a resource, should the operating system grant the request and allocate the resource to the process?

Criteria for Granting or Rejecting the Request:

- The operating system should grant the request and allocate the resource to the process if doing so does not lead to a deadlock in the system.
- The operating system should reject the request and withhold the resource from the process if granting it would cause a deadlock in the system.

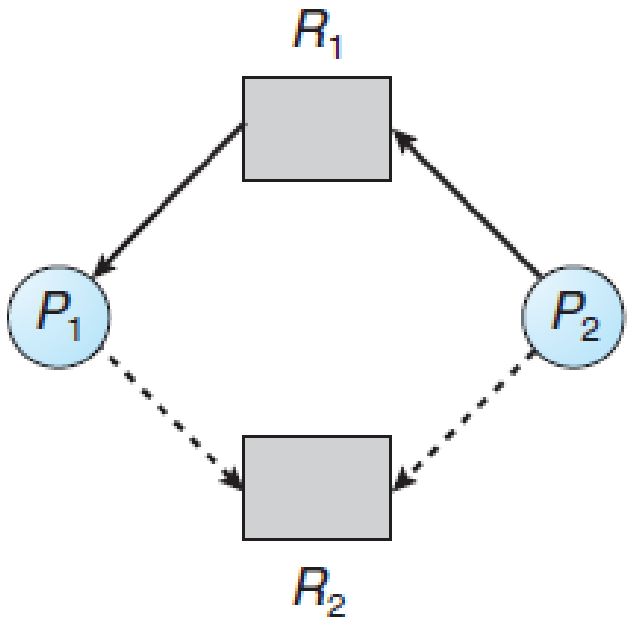
Resource-Allocation Graph for Deadlock Avoidance

If we have a resource-allocation system with only one instance of each resource type, we can use a variant of the resource-allocation graph for deadlock avoidance. The steps are follows:

1. The operating system has the information that which processes will request which resources.
2. We define a *claim edge* $P_i \rightarrow R_j$ indicating that the process P_i will/may request the instance in the resource type R_j. This claim edge is represented by a dotted arrow.
3. We draw all claim edges (from all processes) in the graph.
4. We change a claim edge to the assigned edge:
   - If no deadlock exists, we can assign this resource type to this process and the system is still in a safe state.
   - If a deadlock occurs (unsafe state), we cannot assign this resource type to this process, right now. This process must wait for its request to be satisfied.

Example: Applying Resource-Allocation Graph

Consider the following graph. the process P_1 owns the resource type R_1 while the process P_2 is waiting for the resource type R_1. The situation is that, now, both the processes P_1 and P_2 request to own the resource type R_2. The OS must decide whose process should be own R_2 such that no deadlock happens.



Example: Applying Resource-Allocation Graph

Consider the following two scenarios.

a) First, if the OS assigns the resource type R_2 to the process P_2, the system will be in an <u>unsafe state</u>. Later, if the OS allow the process P_1 request the resource type R_2, the <u>deadlock</u> happens.

Example: Applying Resource-Allocation Graph

Consider the following two scenarios.

b) If the OS assigns the resource type R_2 to the process P_1, the system will be in an <u>safe state</u>. The process P_1 could finish its job, terminate, and return all owned resources to the system. Later, the process P_2 can own the resource types R_1 and R_2. Finally, the process P_2 will finish its job and terminate.

6. Banker's Algorithms

We will apply the Banker's algorithms to solve complicated resource-allocation problems:

- A problem will consist of several processes, resource types, and instances.
- A process might need to own several resource types and instances to finish its task.

What the OS Must Know (Resource Allocation)

1) **Given**: The operating system knows the following information.
   a) *Available*: the number of available/free instances in the system.
   b) *Max*: the number of instances that each process must have to finish its task.
   c) *Allocation*: the number of instances currently owned by each process.
   d) *Need*: the number of instances needed by each process.
   e) *Request*: the number of instances requested by each process.
2) **Banker's Algorithm**: The operating system plans and schedules the allocation of free instances to processes, one by one.
3) **Result**: There are 2 possible results.
   a) Safe state: all processes can eventually finish their tasks and there is a safe sequence.
   b) Deadlock: there is no free/available instance left in the system, all of them have been allocated to the processes. However, some processes could not finish their tasks and are waiting for more instances.

Example: Variables in Banker's Algorithms

Consider a system with five processes P_0 through P_4 and three resource types A, B, and C. Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances. Suppose that, at the starting time, the max number of instances requested and the number of instances allocated to each process can be shown as below. The numbers of instances left are 3, 3, and 2 for A, B, and C, respectively.

- The following variables are given: ***Available***, ***Max***, ***Allocation***.

Available
A B C
3 3 2

	Allocation	*Max*
	A B C	*A B C*
P_0	0 1 0	7 5 3
P_1	2 0 0	3 2 2
P_2	3 0 2	9 0 2
P_3	2 1 1	2 2 2
P_4	0 0 2	4 3 3

Introduction to Banker's Algorithms and Their Types

- Banker's algorithms are used to solve resource-allocation problems involving multiple instances of each resource type.
- This lecture note covers three key Banker's algorithms:
  a) **Safety Algorithm**: Determines whether the system is in a safe state or at risk of deadlock. → Section 7.
  b) **Resource-Request Algorithm**: Helps avoid deadlock by managing resource allocation requests. → Section 8.
  c) **Deadlock-Detection Algorithm**: Detects whether the system is currently in a deadlock state. → Section 9.

Variables in Banker's Algorithms

(m = number of resource types; n = number of processes; i = i-th process; j = j-th resource type)

- ***Available***. A vector (of length m) indicates the number of available/free instances in resource types.
  - ***Available***[j] represents the number of free instances in a resource type R_j.
- ***Max***. An $n \times m$ matrix defines the maximum demand of n processes.
  - ***Max***[i][j] represents the maximum number of instances requested by the process P_i on the resource type R_j.
- ***Allocation***. An $n \times m$ matrix defines the number of resources of each type currently allocated (owned) to n processes.
  - $\boldsymbol{Allocation_i}$ is the vector of the i-th row of the matrix ***Allocation***. It specifies the resources currently allocated to process P_i.
- ***Need***. An $n \times m$ matrix indicates the number of instances needed by n processes.
  - ***Need***[i][j] = ***Max***[i][j] − ***Allocation***[i][j].
  - $\boldsymbol{Need_i}$ is the vector of the i-th row of the matrix ***Need***. It specifies the <u>additional</u> resources that the process P_i may still request to complete its task.

Variables in Banker's Algorithms

- ***Finish***. A vector (of length n) indicates the status of n processes on whether they have done or are waiting for more resources.
  - For example, Finish = [F, T, F, F] means P1 finished its task while P0, P2, P3 do not finish their tasks and are waiting for needed resources.
  - ***Finish***[i] represents the status of process process P_i, which is true or false. The status is true if the process finished its job and terminated. The status is false if the process is still waiting for more resource instances.
- ***Request***. An $n \times m$ matrix defines the number of resources of each type currently requested by n processes.
  - $\boldsymbol{Request_i}$ is the vector of the i-th row of the matrix ***Request***. It specifies the <u>additional</u> resources requested by process P_i.

Vector Inequality

A vector inequality:

- Let X and Y be vectors of length n. We say that $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all $i = 1, 2, \ldots, n$.

Example:

- $[1, 2, 3, 4] \leq [2, 2, 4, 4]$ is True or False?
- $[1, 2, 5, 4] \leq [2, 2, 4, 4]$ is True or False?
- $[10, 2] \leq [15, 5]$ is True or False?
- $[10, 2] \leq [5, 100]$ is True or False?

Example: Variables in Banker's Algorithms

Consider a system with five processes P_0 through P_4 and three resource types A, B, and C. Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances. Suppose that, at the starting time, the max number of instances requested and the number of instances allocated to each process can be shown as below. The numbers of instances left are 3, 3, and 2 for A, B, and C, respectively.

- Find the following variables: ***Available***, ***Max***, ***Allocation***, ***Need***.

Available
A B C
3 3 2

	Allocation	*Max*
	A B C	*A B C*
P_0	0 1 0	7 5 3
P_1	2 0 0	3 2 2
P_2	3 0 2	9 0 2
P_3	2 1 1	2 2 2
P_4	0 0 2	4 3 3

Example: Variables in Banker's Algorithms

7. Safety Algorithm

Problem Statement for Using the Safety Algorithm

Given: A resource allocation problems. The following information is given.

- The numbers of processes and resource types.
- *Available*: the number of available/free instances.
- *Max*: the number of instances that each process must have to finish its task.
- *Allocation*: the number of instances currently owned by each process.
- *Need*: the number of instances needed by each process.

Problem Statement for Using Safety Algorithm:

We would like to determine the state of the system. We use the safety algorithm to:

1) Determine whether the system is in a safe state or experiencing a deadlock state.
2) If the system is in a safe state, identify the safe sequence of process execution.

* Safety Algorithm: Overview *

1) The number of free instances in the system is limited (represented by the variable ***Available***).
2) Each process must own the number of resource instances equal to specific numbers to finish its job (represented by ***Max***).
3) Each process currently owns a certain number of resource instances (represented by the variable ***Allocation***).
4) Each process requires/needs additional instances to complete its task (represented by the variable ***Need = Max - Allocation***).
5) When a process finishes its task, it returns all owned instances to the system, making them available for other processes (these instances are added/updated in the ***Available*** variable).
6) The operating system allocates free instances (from the ***Available*** variable) to processes, one by one, updating the ***Available*** variable accordingly.
7) The safety algorithm is designed to:
   - Determine whether the system is in a safe state or in a deadlock.
   - If the system is in a safe state, identify a safe sequence of process execution.

Safety Algorithm: Safe State and Safe Sequence

- Given a set of processes (requesting resources). A state (situation) is safe if the operating system can allocate resources to each process (up to its maximum) in some order (sequentially) and still avoid a deadlock.
  - There exists a safe sequence (i.e., sequence = processes are executed in this order).
- We call a sequence of processes $\langle \ldots \ldots \ldots \ldots \ldots \ldots \rangle$ is a safe sequence if, for each P_i, the requested resources by P_i can be satisfied by the currently available resources plus the resources held by all previous P_j, with $j < i$.
  - If the resources that P_i needs are not immediately available, then P_i wait until some P_j have finished.
  - Then, P_i can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate.
  - If this sequence goes on until the end without a deadlock, it is a safe sequence.
  - We can find a safe sequence by using the safety algorithm.

Safety Algorithm: Steps

We apply this algorithm to find whether the system is in a safe state and to obtain the safe sequence (the instance allocation order).

Initial: Find ***Available***, ***Max***, ***Allocation***, ***Need***. There are n processes P0 to P(n-1). There are m resource types.

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   $$\boldsymbol{Work} = \boldsymbol{Available} \text{ and } \boldsymbol{Finish}[i] = \textit{false} \text{ for } i = 0, 2, \ldots, n-1.$$

2. Find an index i such that both

   $$\boldsymbol{Finish}[i] == \textit{false} \text{ and } \boldsymbol{Need}_i \leq \boldsymbol{Work}$$

   If no such i exists, go to step 4.

3. Update:

   $$\boldsymbol{Work} = \boldsymbol{Work} + \boldsymbol{Allocation}_i \text{ and } \boldsymbol{Finish}[i] = \textit{true}$$

   Go to step 2.

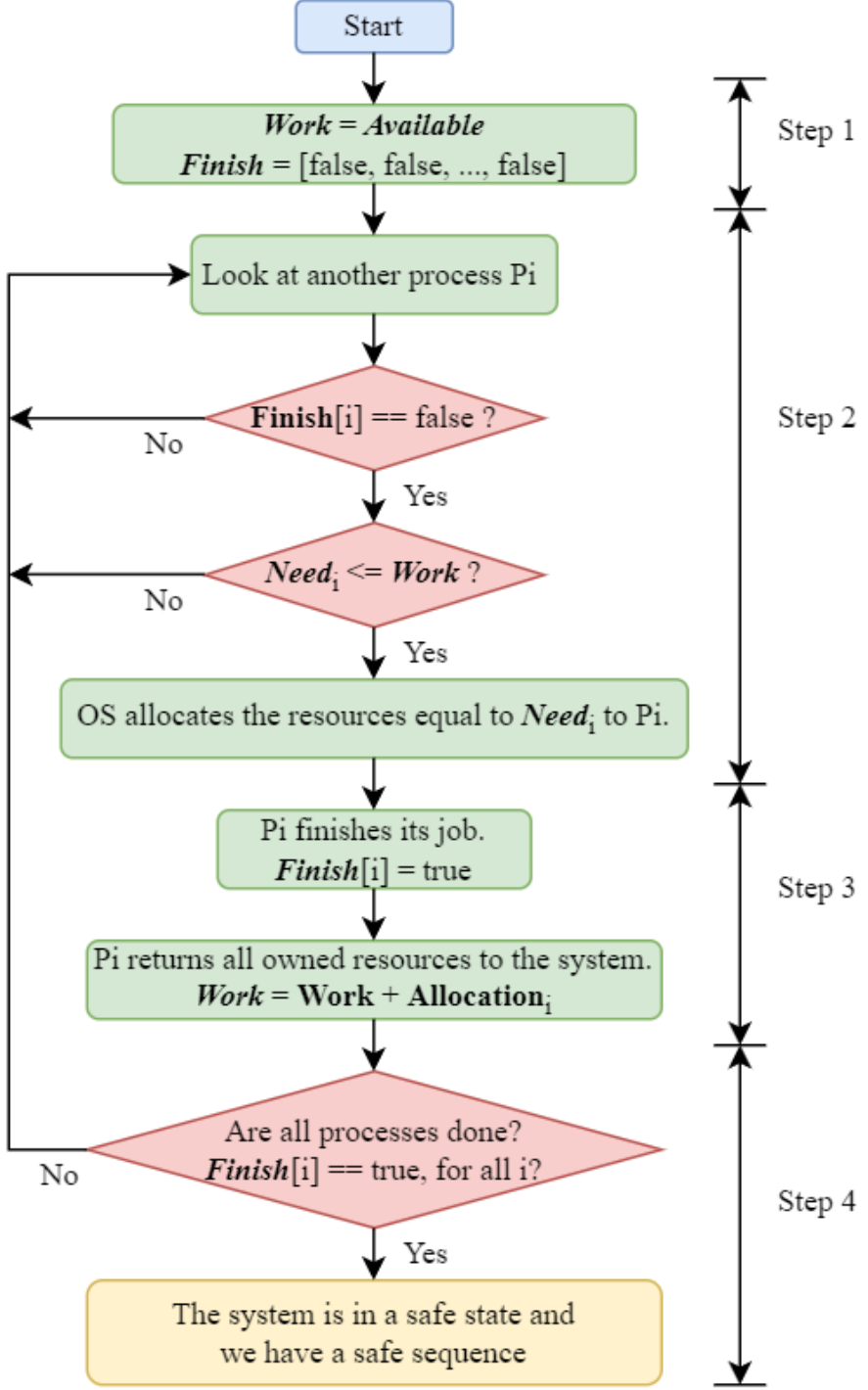
4. If ***Finish***[i] == *true* for all i, then the system is in a <u>safe state</u>. The sequence of i's represents a <u>safe sequence</u>.

Safety Algorithm: Flowchart

The following flowchart show the steps of the safety algorithm.

Step 0:

- There are n processes P0 to P(n-1).
- There are m resource types.
- Find ***Available***, ***Max***, ***Allocation***, ***Need***.



Example: Applying Safety Algorithm

Consider a system with five processes P_0 through P_4 and three resource types A, B, and C. Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances. Suppose that, at the starting time, the max number of instances requested and the number of instances allocated to each process can be shown as below. The numbers of instances left are 3, 3, and 2 for A, B, and C, respectively.

- Determine whether the system is in a safe state. If so, identify a safe sequence.

	Allocation	*Max*
	A B C	*A B C*
P_0	0 1 0	7 5 3
P_1	2 0 0	3 2 2
P_2	3 0 2	9 0 2
P_3	2 1 1	2 2 2
P_4	0 0 2	4 3 3

Available
A B C
3 3 2

8. Resource-Request Algorithm for Deadlock Avoidance

Problem Statement for Using Resource-Request Algorithm and Criteria

Given: A resource allocation problems. The following information is given.

- The numbers of processes and resource types.
- *Available*: the number of available/free instances.
- *Max*: the number of instances that each process must have to finish its task.
- *Allocation*: the number of instances currently owned by each process.
- *Need*: the number of instances needed by each process.
- *Request*: the number of instances requested by each process

Problem Statement for Using Resource-Request Algorithm:

When a process requests a specific number of resource instances, the resource-request algorithm is used to determine whether the operating system should grant or deny the request.

Criteria for Granting or Rejecting the Request:

- The operating system should grant the request and allocate the requested resources if doing so does not lead to a deadlock.
- The operating system should reject the request and withhold the resources if granting them would cause a deadlock.

Resource-Request Algorithm: Steps

During the operation, a process P_i might request an additional instances $\boldsymbol{Request}_i$ (= a vector)

A. We have to revise the following variables: ***Allocation***, ***Available***, and ***Need***:

1) If $\boldsymbol{Request}_i \leq \boldsymbol{Need}_i$, go to step 2). Otherwise, raise an error condition, since the process has exceeded its maximum claim.

2) If $\boldsymbol{Request}_i \leq \boldsymbol{Available}$, go to step 3). Otherwise, P_i must wait, since the resources are not available.

3) Assume that the OS allocates the requested resources to process P_i by modifying the state as follows:

$$\boldsymbol{Available} = \boldsymbol{Available} - \boldsymbol{Request}_i \;;$$

$$\boldsymbol{Allocation}_i = \boldsymbol{Allocation}_i + \boldsymbol{Request}_i \;;$$

$$\boldsymbol{Need}_i = \boldsymbol{Need}_i - \boldsymbol{Request}_i \;;$$

B. We apply the safety algorithm evaluate weather the system is in a safe state or a deadlock.

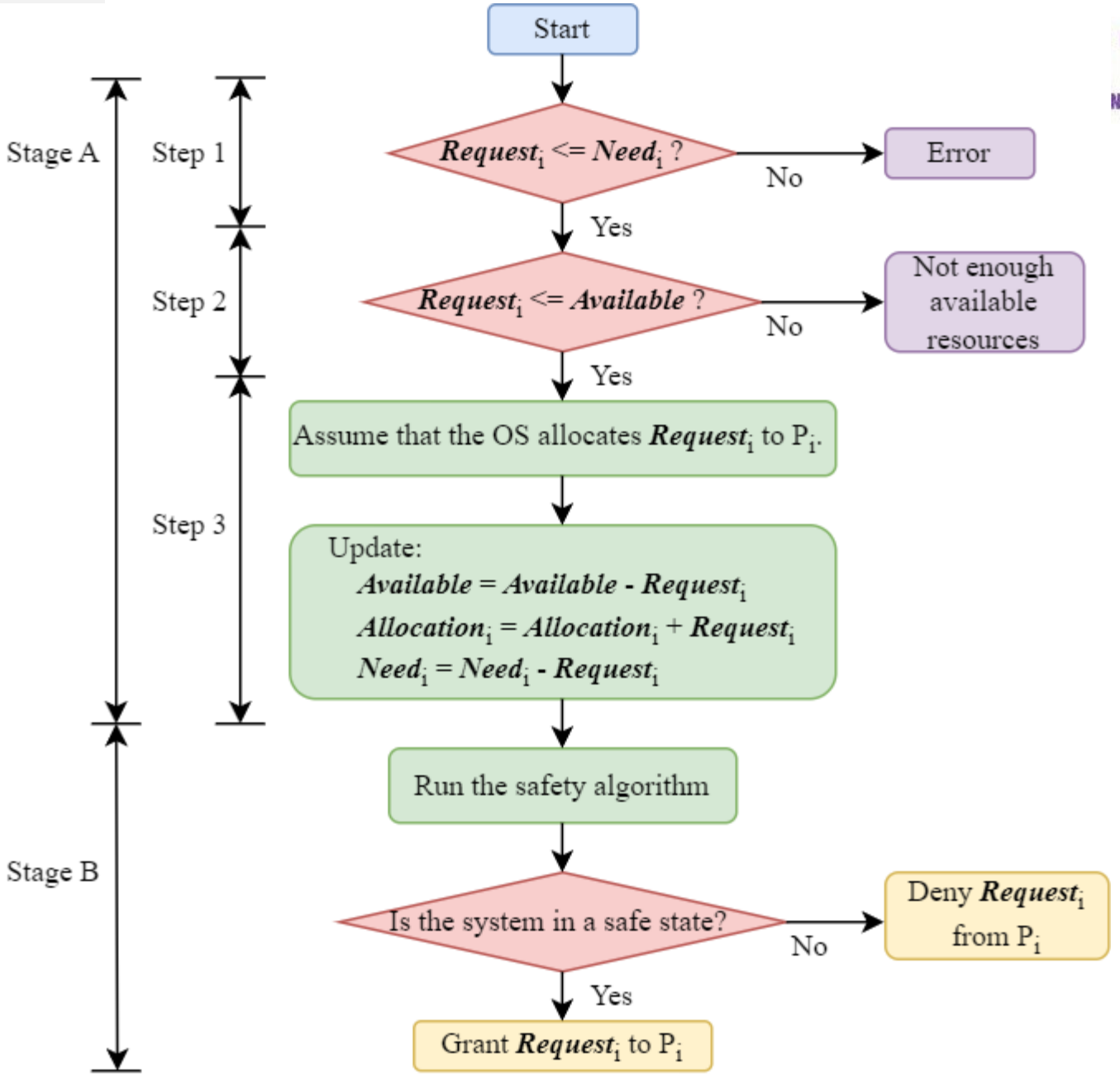
- The OS should grant the request and allocate the resource to the process if doing so does not lead to a deadlock in the system.
- The OS should reject the request and withhold the resource from the process if granting it would cause a deadlock in the system.

Resource-Request Algorithm : Flowchart

The following flowchart show the steps of the resource-request algorithm.

Step 0:

- There are n processes P0 to P(n-1).
- There are m resource types.
- Find ***Available***, ***Max***, ***Allocation***, ***Need***.
- A process P_i requests an additional instances $\boldsymbol{Request}_i$.



Example: Applying Resource-Request Algorithm

According to the previous example (Page 72), if the process P_1 make an additional request $\boldsymbol{Request}_i$ = [1, 0, 2]. If the system allows this request, is the system still in a safe state?

9. Deadlock-Detection Algorithm for Deadlock Detection

If a system does not implement either a deadlock prevention or deadlock avoidance algorithm, a deadlock may occur. In such cases, the system can:

- **Detect** a deadlock using deadlock detection algorithms (covered in this section).
- **Resolve** a deadlock using deadlock recovery techniques (covered in the next section).

Problem Statement for Using the Deadlock-Detection Algorithm

Given: A scenario involving a resource allocation problem including

- the number of available/free resources,
- the number of processes,
- the number of requested instances, (this assumption is new and different from the previous 2 algorithms),
- the number of currently allocated resources.

Problem Statement for Using Safety Algorithm:

In a resource-allocation system where multiple processes request specific resource instances and they will finish their jobs if they own these additional number of resource instances, we would like to determine the state of the system. We use the deadlock-detection algorithm to:

1) Determine whether the system is in a safe state or experiencing a deadlock state.
2) If the system is in a safe state, identify the safe sequence of process execution.

Deadlock-Detection Algorithm

If some resource types have multiple instances, we apply this algorithm to find whether the system contains a deadlock.

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   $$\boldsymbol{Work} = \boldsymbol{Available}$$

   For i = 0, 1, …, n-1,
   - if $\boldsymbol{Allocation}_i = 0$, then ***Finish***[i] = *true*.
   - if $\boldsymbol{Allocation}_i \neq 0$, then ***Finish***[i] = *false*.
2. Find an index i such that both: ***Finish***[i] = *false* and $\boldsymbol{Request}_i \leq \boldsymbol{Work}$

   If no such i exists, go to step 4.
3. Update: $\boldsymbol{Work} = \boldsymbol{Work} + \boldsymbol{Allocation}_i$ and ***Finish***[i] = *true*

   Go to step 2.
4. If ***Finish***[i] = *false* for some i, → the process P_i is deadlocked and, then, the system is in a deadlocked state.

Note: Similar to the safety algorithm. However, the differences are:
- define the initial values in Finish,
- This condition where in the safety algorithm, we use

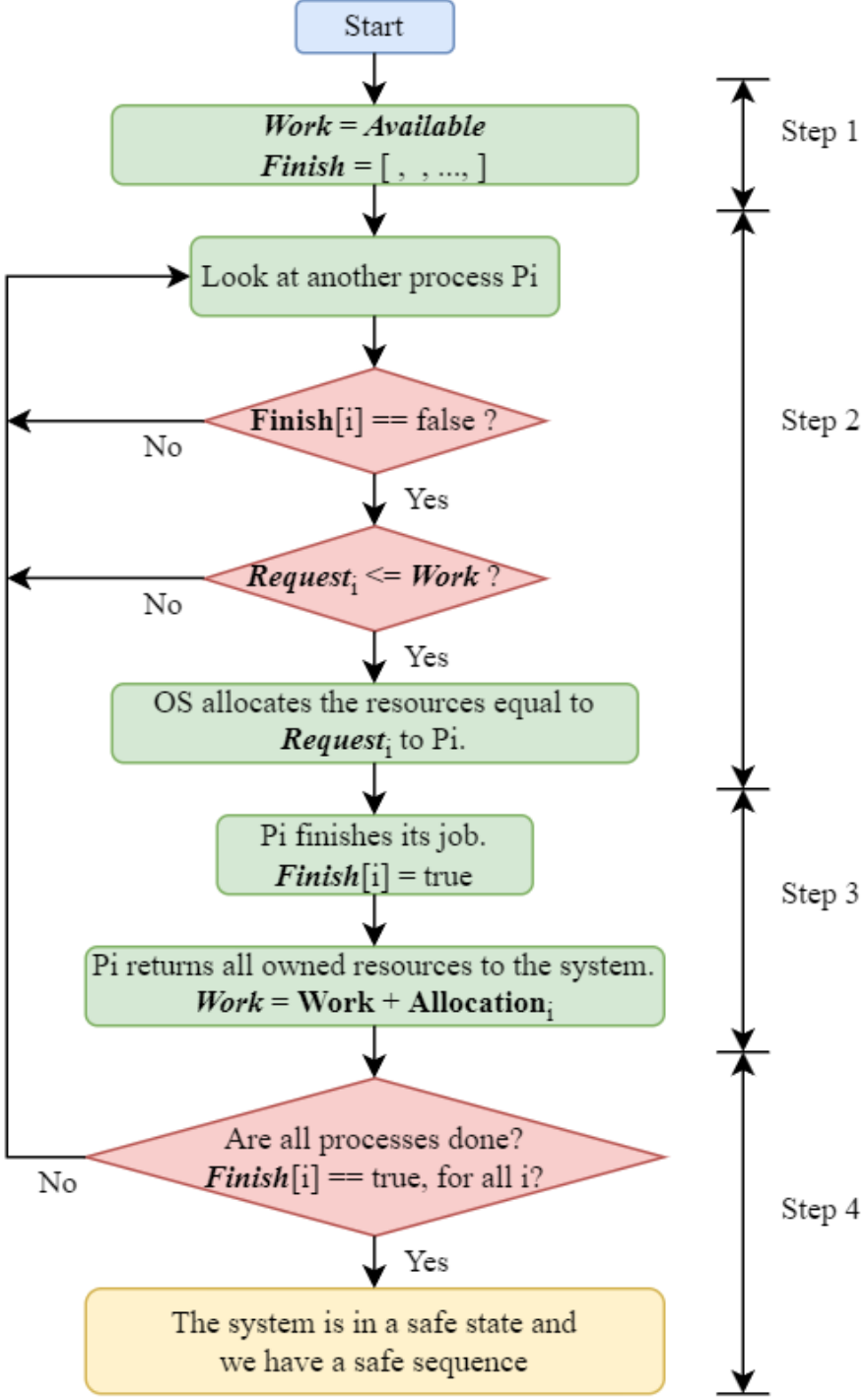
  $$\boldsymbol{Need}_i <= \boldsymbol{Work}$$

Deadlock-Detection Algorithm: Flowchart

The following flowchart show the steps of the deadlock-detection algorithm.

Step 0:

- There are *n* processes P0 to P(*n*-1).
- There are *m* resource types.
- Find ***Available***, ***Allocation***, ***Request***.



Example: Applying the Deadlock-Detection Algorithm

Consider a system with five processes P_0 through P_4 and three resource types A, B, and C. Resource type A has seven instances, resource type B has two instances, and resource type C has six instances. Suppose that, at the starting time, we have the following resource-allocation state:

	Allocation		*Request*	*Available*
	A B C		*A B C*	*A B C*
P_0	0 1 0	P_0	0 0 0	0 0 0
P_1	2 0 0	P_1	2 0 2	
P_2	3 0 3	P_2	0 0 1	
P_3	2 1 1	P_3	1 0 0	
P_4	0 0 2	P_4	0 0 2	

Determine whether a deadlock will occur in the system.

Trade-Offs in Using the Deadlock Detection Algorithm

Running a deadlock detection algorithm incurs significant computational overhead. Therefore, the decision on when to invoke it must be carefully considered.

- If deadlocks occur frequently, the algorithm may need to be run more often.
- When a deadlock occurs, the number of processes involved in the deadlock cycle may increase over time. Detecting it early helps minimize system disruption and potential damage.

10. Recovery from Deadlock

When a detection algorithm determines that a deadlock exists, there are two approaches to recover the system by eliminating the deadlock:

1) Process termination
2) Resource preemption

1. Process Termination

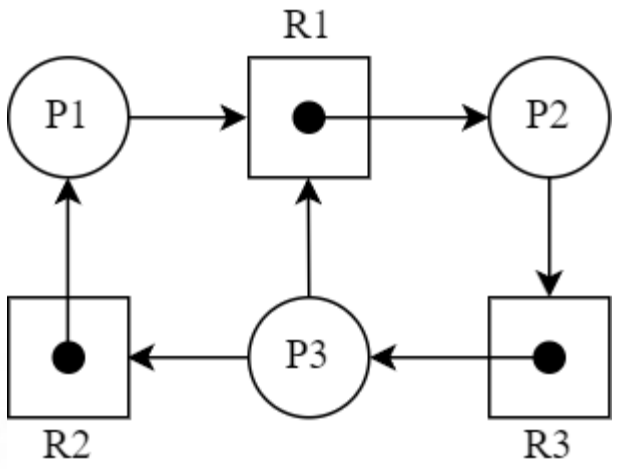
To eliminate deadlocks by the *process termination*, we use one of two methods.

a) Abort all deadlocked processes. Note that by using this method, if a process have computed for a long time, it has to restart again since the beginning.

b) Abort one process at a time until the deadlock cycle is eliminated. However, this method incurs considerable overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked.

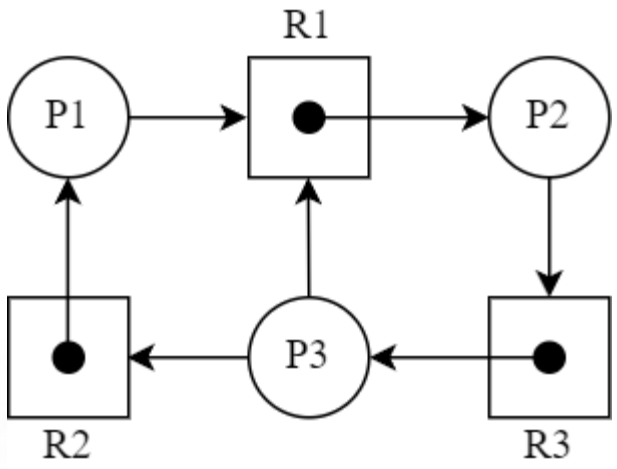
Example: Abort all deadlocked processes

Solve the following deadlock state by aborting all deadlocked processes.



Example: Abort one process at a time

Solve the following deadlock state by aborting one process at a time.



2. Resource Preemption

To eliminate deadlocks using *resource preemption*, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken. Note that then three issues need to be considered:

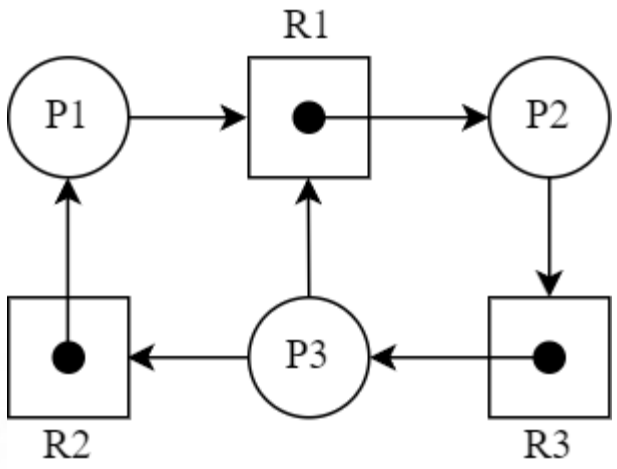
a) Select a victim process.

b) Role back the victim process to a safe state.

c) Ensure that starvation will not occur on the same processes.

Example: Resource preemption

Solve the following deadlock state by resource preemption.



Reading Assignment

- Section 7.1, 7.2, 7.3, 7.4, 7.5, 7.6, 7.7

Glossary (by Google NotebookLM)

- **Resource Allocation**: The process by which an operating system distributes and manages system resources among competing processes.
- **Deadlock**: A situation where two or more processes are blocked indefinitely, each waiting for a resource held by another process in the set.
- **Safe State**: A system state in which there exists a sequence of process executions that allows every process to run to completion, even if they have to wait for resources held by preceding processes.
- **Safe Sequence**: An ordering of processes such that for each process in the sequence, the resources it will still need can be satisfied by the currently available resources plus the resources held by all the processes preceding it in the sequence.
- **Mutual Exclusion**: A condition where only one process can use a non-sharable resource at any given time.
- **Hold and Wait**: A condition where a process holds at least one resource and is waiting to acquire additional resources held by other processes.
- **No Preemption**: A condition where a resource can only be released voluntarily by the process holding it, after that process has completed its task.

Glossary (by Google NotebookLM)

- **Circular Wait**: A condition where a set of processes are waiting for each other in a circular chain, with each process waiting for a resource held by the next process in the chain.
- **Resource-Allocation Graph**: A directed graph used to model the allocation of resources to processes, with processes represented by circles and resource types (with instances) by rectangles.
- **Request Edge**: A directed edge in a resource-allocation graph from a process to a resource type, indicating that the process has requested an instance of that resource type.
- **Assignment Edge**: A directed edge in a resource-allocation graph from an instance of a resource type to a process, indicating that the resource instance has been allocated to the process.
- **Wait-For Graph**: A graph used for deadlock detection (in systems with single instances of resources) where nodes represent processes, and a directed edge from P1 to P2 means P1 is waiting for a resource held by P2.
- **Banker's Algorithm**: A deadlock avoidance algorithm that requires the operating system to know the maximum resource needs of each process in advance and simulates resource allocation to ensure a safe state is always maintained.

Glossary (by Google NotebookLM)

- **Safety Algorithm**: A component of the Banker's Algorithm used to determine if the current state of the system is safe by checking if a safe sequence exists.
- **Resource-Request Algorithm**: A component of the Banker's Algorithm used to decide whether a request for resources by a process can be granted without leading the system into an unsafe state.
- **Deadlock Detection**: The process of identifying whether a deadlock has occurred in the system.
- **Deadlock Recovery**: The process of resolving a deadlock once it has been detected, typically through process termination or resource preemption.
- **Process Termination**: A method of deadlock recovery that involves aborting one or more of the deadlocked processes.
- **Resource Preemption**: A method of deadlock recovery that involves forcibly taking away resources from one or more deadlocked processes and allocating them to other processes to break the deadlock.